Universidade Cruzeiro do Sul
Ciência da Computação

Aula 4 – Introdução à Sintaxe das Linguagens de Programação

Paradigmas de Linguagens de Programação

Prof. Álvaro Colombero Prado

Objetivos



- Revisar os conceitos de sintaxe e semântica. Diferenciar sintaxe léxica de sintaxe concreta.
- Compreender para que serve a gramática de uma linguagem de programação.
- Definir para que servem os nomes em uma linguagem de programação.

Introdução

Pode-se definir uma linguagem de programação como um conjunto de termos e regras **sintáticas** e **semânticas** que permitem a construção de instruções processáveis por um computador.

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    double A[][] = new double[5][4];
    double soma = 0;
    for (int i=0;i<5;i++){
        for (int j=0;j<4;j++){
            System.out.print("Digite o elem
            A[i][j] = Double.valueOf(input.
            soma = soma + A[i][j];
        }
    }
```

Introdução

- A **sintaxe** de uma linguagem de programação é a forma de suas expressões, sentenças e unidades de programa. Sua **semântica** é o significado dessas expressões e unidades de programa (SEBESTA, 2010 p. 136).
- A sintaxe de uma linguagem é descrita por um conjunto de regras e ela deve ser entendida por:
  - Quem está criando-a.
  - Os avaliadores iniciais.
  - Os implementadores da linguagem.
  - Os usuários da linguagem.

Sintaxe

Qual a melhor forma para descrever (especificar) a sintaxe de linguagem? Linguagem natural ou formal?

Métodos formais tem sido usados desde o Algol na década de 60 para descrever a sintaxe das linguagens.



Sintaxe

- Um código-fonte é apenas uma grande cadeia de caracteres.
- A sintaxe de uma linguagem é definida por meio de métodos formais:
  - Expressões Regulares.
  - Autômatos Finitos
  - Gramáticas Livres de Contexto.
- Gramática é uma metalinguagem que visa definir as cadeias de caracteres que podem formar um programa sintaticamente correto.

Sintaxe

- Níveis de sintaxe:
  - **Sintaxe léxica**: regras para os símbolos básicos, como identificadores, literais, operadores e pontuação.
  - **Sintaxe concreta**: é a representação real do programa usando símbolos léxicos como seu alfabeto.
  - **Sintaxe abstrata**: considera apenas as informações essenciais, sem se preocupar com detalhes como pontuação e parênteses, por exemplo.

Sintaxe

- **Sintaxe léxica**: determina como os caracteres (símbolos de um alfabeto) podem ser agrupados para formar elementos válidos da linguagem:
  - **Lexema:** sequência de caracteres do programa fonte.
  - **Token:** classificação dos lexemas que se encaixam em determinado padrão.
  - **Padrão:** descrição da forma que os lexemas podem assumir.

Sintaxe

- Na linguagem C os tokens são divididos em seis grupos:

Lexema	Grupo do token
n, soma1, main, strcpy	identificador
int, float, if, for, return	palavra-chave ou reservada
10, 12.5, -5	constante
“Digite um número”	string
= & ( ) [ ] { } ;	símbolos especiais
+ - * / % ++ – == > < !=	operadores

Sintaxe

- **Sintaxe concreta**: determina como os tokens definidos pela sintaxe léxica podem ser agrupados para criar estruturas (comandos) válidos.
- Exemplos:
  - `soma = x + 10;`
  - `if (x > 10) {soma = 0;}`
  - `x = (y + 10 * z;`

Sintaxe

- Tamanhos das gramáticas das diferentes linguagens:
  - Pascal – 5 páginas.
  - C – 6 páginas.
  - C++ - 22 páginas.
  - Java – 14 páginas.
- O processo de construção de gramáticas e a sua utilização para analisar a sintaxe de programas será visto nas disciplinas de Teoria da Computação (5ª fase) e Compiladores (6ª fase).

Nomes e Identificadores

Nomes e Tipos de Dados

- Nomes ou identificadores, são utilizados em programas para identificar variáveis, tipos definidos pelo usuário, funções, etc.
- Quais são os identificadores do exemplo abaixo?

```
int quantidade;
int soma(int a, int b){
   return a + b;
}
```

Nomes e Tipos de Dados

- Para criar os identificadores, todas as linguagens devem responder às perguntas abaixo
  - Quais as regras para formação dos nomes?
  - Os nomes são sensíveis à capitalização (case sensitive)?
  - Os termos especiais são palavras reservadas ou palavras-chave?
    - Palavras reservadas possuem um significado especial e não podem ser usadas como identificadores.
    - Palavras-chave possuem significado especial apenas em contextos específicos e podem ser usadas em outros contextos como identificadores (Fortran).

Nomes e Tipos de Dados

- Regras léxicas definem como os nomes podem ser formados:
  - Em geral inicia com uma letra e possui uma sequência de letras ou dígitos.
  - Algumas linguagens permitem o uso de símbolos especiais (_) na definição dos nomes dos identificadores e outras não.
  - PHP - nomes de variáveis iniciam com $;
    - `$nome = "IFSC";`
    - `echo $nome;`

Nomes e Tipos de Dados

- Perl - $ (escalar), @ (vetor) ou % (hash) no início do nome especificam o seu tipo:
  - `@cidades = ("Lages", "Florianópolis", "Correia Pinto");`
  - `print "$cidades[0]\n";`
  - `print "$cidades[1]\n";`
  - `print "$cidades[2]\n";`
- Ruby - @ ou @@ indicam se é uma variável de instância ou variável de classe.

Nomes e Tipos de Dados

- As primeiras linguagens só usavam letras maiúsculas e possuíam restrição quanto ao tamanho do nome – Seis caracteres no caso do FORTRAN.
  - O FORTRAN95 restringe a 31 caracteres seus nomes.
  - C não tem restrição para os nomes internos, mas apenas os 63 primeiros caracteres são significativos e nomes externos também são restritos a 31 caracteres;
  - Java, C# e Ada não tem limitação de tamanho.

Nomes e Tipos de Dados

- Pascal e Ada não diferenciam entre maiúsculas e minúsculas, C, C++ e Java sim. Quais as vantagens e desvantagens de fazer esta diferenciação?
  - Força a padronização (+).
  - Dificulta a escrita (-).
  - Prejudica a legibilidade (-)???
  - Permite a associação de nomes semelhantes com diferentes capitalizações (+ ou -)???

Nomes e Tipos de Dados

Nomes são utilizados para identificar variáveis. O que é uma variável?

Uma variável é uma abstração de uma ou mais células de memória identificada por um nome.





Sintaxe

- Exemplo 1: Implemente o programa abaixo em C, execute o programa três vezes, anote e compare os resultados apresentados na tela.

```
#include <stdio.h>

int main(void) {

    int x=2147483647;

    printf("%d \n", x);

    printf("%ld \n", &x);

    return 0;

}
```

Bibliografia

- Material complementar:

TUCKER, A. B.; NOONAN, R. E.; **Linguagens de programação: princípios e paradigmas.** 2. ed. São Paulo: McGrall-Hill, 2009. Seções 4.1 e 4.2.

SEBESTA, R. W.; **Conceitos de linguagens de programação**. 9. ed. Porto Alegre: Bookman, 2011. Seções 3.1, 3.2, 5.1 e 5.2.